

Python Programming with Artificial Intelligence

Advanced Python Fundamentals

Chengwei QIN @ AI Thrust



Classes and Objects

- OOP, Constructor/Destructor, Class/Object-level attributes Dunder Methods

Inheritance

- Subclass/Superclass, Polymorphism

Modules and Packages

- built-in/user-defined modules, import, `__name__ == '__main__'`, packages

Advanced Iteration: Recursion & Generators

- yield statement



Classes and Objects

Object-Oriented Programming

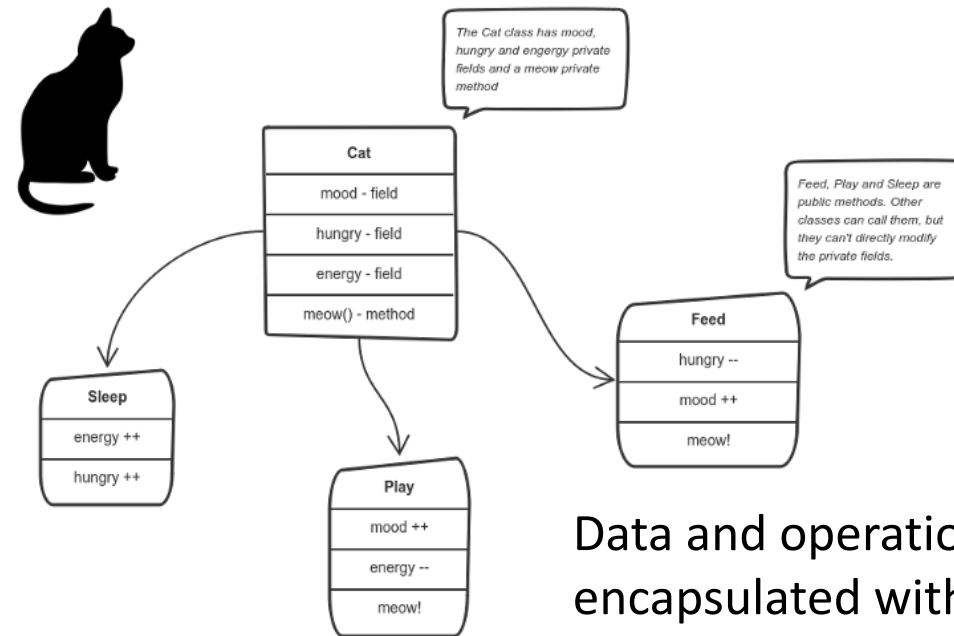
Two main paradigms in programming:

- Procedural Programming
- Object-Oriented Programming (OOP)

Core idea of OOP: **encapsulate** both data and operations within *objects* and design a program as a collection of interacting *objects*.

Example for OOP:

- There was a person named *Alice*
- *Alice adopted* a cat *Tom*
- *Tom meowed* for hungry
- *Alice bought a fish*
- *Alice fed Tom* with *a fish*
- *Tom slept* for 10 minutes



Data and operations
encapsulated within a cat

Classes and Objects

A **Class** defines abstract characteristics of **objects**, specifying their attribute and methods

- **Attributes:** Represent the properties or data associated with objects of the class
- **Methods:** Define the behaviors or actions that objects of the class can perform (*involve other objects, dependent on attribute values, modify attributes*)

Human

- Attributes: name, sanity, health, age, money
- Methods: adopt a pet, feed a cat, buy food, eat, play, drive

Cat

- Attributes: hunger level, thirst level, energy, happiness
- Methods: meow, eat, sleep

Fish

- Attributes: size, type, price
- Methods: swim

Example for OOP:

- There was a person named *Alice*
- *Alice* *adopted* a cat *Tom*
- *Tom* *meowed* for hungry
- *Alice* *bought* a fish
- *Alice* *fed* *Tom* with *the fish*
- *Tom* *slept* for 10 minutes

Classes and Objects

An **object** is an **independent instance** of a class

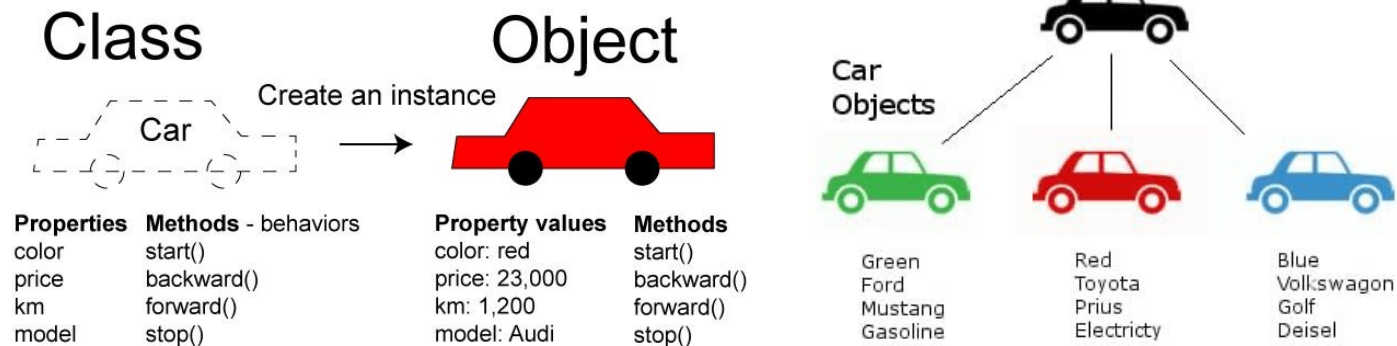
- A class is like a blueprint for building sth., while an instance (also called object) is the thing built according to the blueprint.
- Each object stores its own specific data for the attributes
- Each object can call the methods defined in its class

Class:

- Human
- Cat
- Fish

Objects:

- Alice
- Tom
- the fish that Tom ate



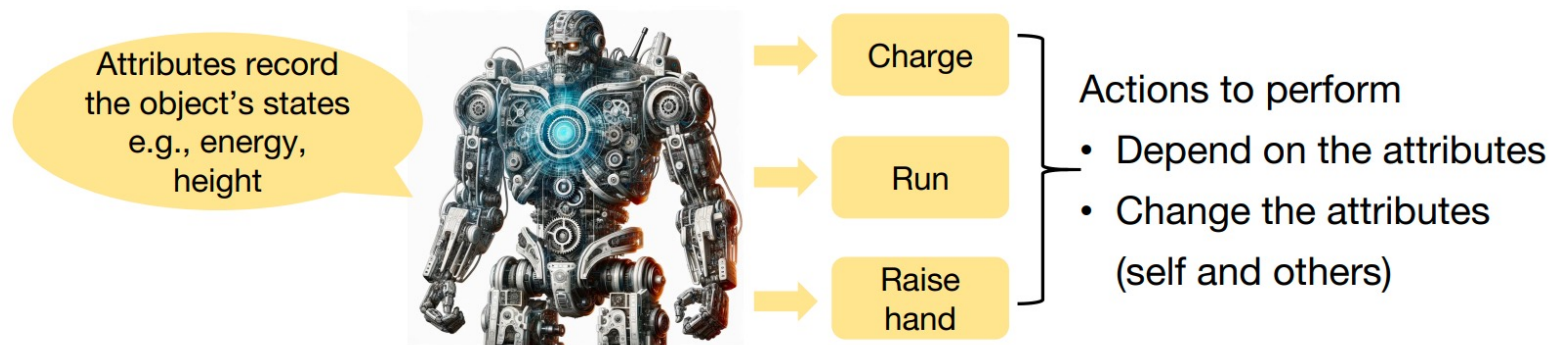
Example for OOP:

- There was a person named *Alice*
- *Alice adopted* a cat *Tom*
- *Tom meowed* for hungry
- *Alice bought a fish*
- *Alice fed Tom* with *the fish*
- *Tom slept* for 10 minutes

Encapsulation

A fundamental principle of OOP: pack data (attributes) and functions (methods) together inside an object.

- **Modularity:** keep complex programs organized and easier to understand
- **Reusability:** Hides implementation details and allowing objects to be reused as black boxes
- **Safety:** Protects internal state by restricting direct access, reducing the chance of misuse



In Python, everything is an object, e.g., Datatypes, functions, exceptions

- built-in types such as list, tuple, string, dictionary, set, number, NoneType, Boolean, ...
- Each function is an object
- Each raise exception is an object
- And more

Simple Class Definition

Syntax:

```
01. class ClassName:
02.     # Statements to define attributes
03.     # Statements to define methods
```

```
class MyClass:
    x = 5
✓ 0.0s
```

Create an object

The class name is used as a function call to create an object

- Attributes defined in a class are shared across all the objects

```
p1 = MyClass()
print(p1.x)
✓ 0.0s
5
```

Use the “pass” statement as a **placeholder** when defining a class without specifying its attributes or methods yet

```
24 class AITrust:
25     pass
```

Class Method and Object Method

A **class method** operates on the class itself, while an **object method** operates on a specific object created from that class.

```
class Student:
    school_name = "Peking U"

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name

    #object method
    def change_sch(self, new_name):
        self.school_name = new_name
        return
```

```
# Object method only changes the object
Mary = Student()
Mary.change_sch("HKUST(GZ)")
Mary.school_name
```

```
'HKUST(GZ)'
```

```
# The class attribute is not changed
Jack = Student()
Jack.school_name
```

```
'Peking U'
```

```
# class method changes the class
Student.change_school("HKUST(GZ)")
Jack = Student()
Jack.school_name
```

```
'HKUST(GZ)'
```

Class Method Definition

Syntax (Similar to defining a regular function)

```
@classmethod
def function_name(cls, param1, param2, ...):
    # put method statements here
```

- first parameter: represents the **class** itself and allows access to its attributes and methods
 - It can technically be named anything, but by convention we use **cls**
 - It is automatically and **implicitly** passed when the method is called

```
# Class method
class Student:
    # Class attribute shared by all instances
    school_name = "High School"

    #Class Method
    @classmethod
    def change_school(cls, new_name):
        # Uses 'cls' to modify the shared class attribute
        cls.school_name = new_name
```

Object Method Definition

Syntax (Similar to defining a regular function)

```
def function_name(self, param1, param2):  
    # put method statements here
```

- first parameter: represents the **object** itself and allows access to its attributes and methods
 - It can technically be named anything, but by convention we use **self**
 - It is automatically and **implicitly** passed when the method is called

```
class Human:  
    age = 10  
    def print_age(self):  
        print(self.age)
```

```
student_1 = Human()  
student_1.print_age()
```

The object `student_1` is automatically passed to the method of `student_1.print_age()`

Any User-Created Class Is a Subclass of `object`

- `object` is a built-in class in Python. Any class, including the user-created class, in Python is a subclass of it.

```
[5]: class Person:
      pass

      issubclass(Person, object)
```

```
[5]: True
```

- `Person` inherits attributes and methods of `object`

```
[12]: dir(object)
```

```
[12]: ['__class__',
      '__delattr__',
      '__dir__',
      '__doc__',
      '__eq__',
      '__format__',
      '__ge__',
      '__getattribute__']
```

```
[11]: class Person:
      pass
```

```
      dir(Person)
```

```
[11]: ['__class__',
      '__delattr__',
      '__dict__',
      '__dir__',
      '__doc__',
      '__eq__',
```

- We use `object` to represent the built-in class in Python.
- We use `object` to refer to an instance of a class.

Constructor `__new__()`

`__new__()`

- any class inherits this method from `object`
- it is automatically called to create an instance (memory allocated) of the class.

```
[16]: class Person:  
      pass  
  
Mike = Person() #__new__ is called and the instance Mike is created
```

■ ■ ■ Initializer `__init__`

`__init__()`: is automatically and implicitly called when an object is created.

- The default `__init__()` refers to the one directed inherited from `object`.
- Although `__init__` is in fact an initializer rather than a constructor, it is often called a **constructor**.

```
•[16]: class Person:
        pass

Mike = Person() #__new__() is called and the instance Mike is created;
            #__init__() is called and it does nothing here since __init__() is not overloaded when defining Person;
```

Override `__init__()`

You can re-define (override) `__init__()` to add attributes to the created object.

```
[26]: class Person:
      age = 10
      def __init__(self, wt):
          self.weight = wt

      Jack = Person(wt=100) #1. __new__ allocates memory and returns a brand-new instance of Person.
                           #2. __init__ creates Jack.weight and assigns 100 to it
      print("Jack age:", Jack.age) # Python looks for age first from Jack (fails), and then from Person
      print("Jack weight:", Jack.weight)
```

```
Jack age: 10
Jack weight: 100
```

Attributes can be created dynamically during a method call

```
class Human:
    age = 10

    def set_name(self, name_x):
        self.name = name_x
```

```
student_1 = Human()
print(student_1.name)
```

⊗ ✨ 0.0s

AttributeError Traceback (most recent call last)

Cell In[9], line 2

```
1 student_1 = Human()
----> 2 print(student_1.name)
```

AttributeError: 'Human' object has no attribute 'name'

```
student_1.set_name('John')
print(student_1.name)
```

John

Attributes

Two types of attributes in Python classes

- **Class-level attributes:** shared by all objects of the class
- **Object-level attributes:** created for each individual object, often inside methods such as `__init__()`

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Class-level

Object-level

Attributes

Two types of attributes

- **Class-level attributes**: shared by all the objects
- **Object-level attributes**: created for each individual object, often inside methods such as `__init__()`

```
class Human:
    location = 'Guangzhou'
    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Class-level

Object-level

```
alice = Human('Alice', 25)
bob = Human('Bob', 30)
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Guangzhou Guangzhou
4345001776 4345001776

```
Human.location = 'Hong Kong'
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Hong Kong Hong Kong
4836193328 4836193328

Change class-level attributes for all the objects at a time

```
alice.location = 'Hong Kong'
print(alice.location, bob.location)
print(id(alice.location), id(bob.location))
```

✓ 0.0s

Hong Kong Guangzhou
4836171440 4345001776

Change class-level attributes for one object will not influence the others

Attributes

Two types of attributes

- **Class-level attributes:** shared by all the objects
- **Object-level attributes:** created for each individual object, often inside methods such as `__init__()`

Special case: Mutable class attributes

- If a class attribute refers to a mutable object (e.g., a list or dictionary), changes made through one instance will affect all instances that share it

```
class Human:
    location = 'Guangzhou'
    lst = []
    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=20)
print(alice.lst, bob.lst)
print(id(alice.lst), id(bob.lst))
alice.lst.append(1)
print(alice.lst, bob.lst)
```

✓ 0.0s

```
[] []
4836004224 4836004224
[1] [1]
```

Attributes

Two types of attributes

- **Class-level attributes:** shared by all the objects
- **Object-level attributes:** created for each individual object, often inside methods such as `__init__()`

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Class-level

Object-level

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=23)
print(alice.age, bob.age)
print(id(alice.age), id(bob.age))
```

✓ 0.0s

22 23
4344769488 4344769520

Created during
initialization

```
alice.height = 162
print(alice.height)
```

✓ 0.0s

Python

162

Created manually

```
print(bob.height)
```

✗ 0.0s

Python

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[43], line 1
----> 1 print(bob.height)

AttributeError: 'Human' object has no attribute 'height'
```

No influence on each other

Delete

Remove a **specific attribute** from an object, or delete the **entire object**.

- use the “del” keyword

```
1 Joe = Person('Joe', 18, "HKUST(GZ)-5B")
2 print(Joe.age)
3 del Joe.age
4 print(Joe.age)
```

[11] 0.0s Python

... 18

... -----

AttributeError Traceback (most recent call last)

Cell In[11], line 4

2 print(Joe.age)

3 del Joe.age

----> 4 print(Joe.age)

AttributeError: 'Person' object has no attribute 'age'

delete attribute

```
1 Joe = Person('Joe', 18, "HKUST(GZ)-5B")
2 print(Joe.age)
3 del Joe
4 print(Joe.age)
```

[12] 0.0s Python

... 18

... -----

NameError Traceback (most recent call last)

Cell In[12], line 4

2 print(Joe.age)

3 del Joe

----> 4 print(Joe.age)

NameError: name 'Joe' is not defined

delete object

■ Destructor

`__del__()`: automatically called when an object is destroyed

- **default destructor**: does nothing by default
- **customized destructor**: defines specific actions to perform when the object is destroyed

Automatic garbage collection: an object is destroyed when all references to it are deleted

```
1 class Human:
2     _location = 'Guangzhou'
3
4     def introduce(self):
5         print("I am {} from {}. I am {} years old. "\
6             .format(self.name, self.location,
7                 self.age))
8
9     def __init__(self, name, age):
10        self.name = name
11        self.age = age
12        print('object created')
13
14    def __del__(self):
15        print('object deleted')
```

```
17  alice = Human(name='Alice', age=22)
18  del alice
19  print('hello')
```

```
object created
object deleted
hello
```

Dunder ("double underscore") **methods** are special methods that Python runs automatically (i.e., you don't need to call them) when you use built-in features or symbols.

- `__new__`: constructor
- `__init__`: initializer
- `__del__`: destructor
- `__lt__`, `__gt__`, `__eq__`, `__ge__`, `__le__`: comparison operations with `<`, `>`, `==`, `<=`, `>=`
- `__add__`, `__sub__`, `__mul__`, `__truediv__`, `__mod__`: arithmetic operations
- `__str__`: defines the string representation of the object (used by `str()`)
- `__getitem__`: supports indexing with `[i]`
- `__len__`: defines behavior of `len(obj)`
- `__getattr__`: called when an undefined attribute is accessed
- `__setattr__`: called when setting an attribute
- `__delattr__`: called when deleting an attribute

Examples of Dunder Methods

- When you use a symbol (+): Writing `a + b` automatically triggers `a.__add__(b)`.
- When you use a built-in function (`len()`): Writing `len(my_list)` automatically triggers `my_list.__len__()`.
- When you create an object (`Person()`): Writing `p = Person()` automatically triggers `Person.__new__()` and `Person.__init__()`.

Overload __str__

The `__str__()` method is a method defined in `object`. Hence, every user-defined class inherits this method from `object`. When an object is passed to `print()`, Python uses its `__str__()` method to obtain a human-readable string representation.

```
class Human:
    location = 'Guangzhou'
    lst = []
    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return "Name:{} Location: {} Age: {}".format(self.name, self.location, self.age)
```

```
alice = Human(name='Alice', age=22)
print(alice)
```

✓ 0.0s

<__main__.Human object at 0x106808100>

Before overloading: default string

```
alice = Human(name='Alice', age=22)
print(alice)
```

✓ 0.0s

Name:Alice Location: Guangzhou Age: 22

After overloading

Overload the '<' operator with the __lt__() method

def __lt__(self, other)

- Compares two objects and returns True if self < other, otherwise False

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __lt__(self, other):
        return self.age < other.age
```

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=20)
print(alice < bob)
```

⊗ 0.0s

Without overloading

TypeError Traceback (most recent call last)
Cell In[60], line 3

```
1 alice = Human(name='Alice', age=22)
2 bob = Human(name='Bob', age=20)
----> 3 print(alice < bob)
```

TypeError: '<' not supported between instances of 'Human' and 'Human'

```
alice = Human(name='Alice', age=22)
bob = Human(name='Bob', age=25)
print(alice < bob)
```

✓ 0.0s

After overloading

Python

True



The `__lt__()` method is defined in the `object` class, so every user-defined class inherits a default `__lt__()` method.

When we use `<` operator to compare two objects, Python attempts to use `__lt__()` to perform the comparison.

The default method does **not** provide a comparison rule for Human objects. So we need to override `__lt__()` to define what `<` means for objects of our class. In this example, we compare `self.age` with `other.age`

Overload the build-in len() function using the __len__() method

def __len__(self)

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(len(team))
```

⊗ ✨ 0.0s

Without overloading

TypeError Traceback (most recent call last)

Cell In[75], line 4

```
2 team.add_member('Alice')
3 team.add_member('Bob')
----> 4 print(len(team))
```

TypeError: object of type 'Team' has no len()

```
class Team:
    def __init__(self):
        self.members = []

    def add_member(self, name):
        self.members.append(name)

    def __len__(self):
        return len(self.members)
```

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(len(team))
```

✓ 0.0s

2

After overloading

__getitem__

Overload the indexing operation [i] using the `__getitem__()` method

def __getitem__(self, i):

- Takes an index as an argument and returns the corresponding element

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(team[1])
```

⊗ ✨ 0.0s

Without overloading

```
-----
TypeError                                Traceback (most recent call last)
Cell In[79], line 4
      2 team.add_member('Alice')
      3 team.add_member('Bob')
----> 4 print(team[1])

TypeError: 'Team' object is not subscriptable
```

```
class Team:
    def __init__(self):
        self.members = []

    def add_member(self, name):
        self.members.append(name)

    def __len__(self):
        return len(self.members)

    def __getitem__(self, i):
        return self.members[i]
```

```
team = Team()
team.add_member('Alice')
team.add_member('Bob')
print(team[1])
```

✓ 0.0s

After overloading

Bob

Task: Build a simple library management system with the ability to add, check out, and return books.

Book Class

- Attributes: ID, title, author, a state indicating whether the book is borrowed or not
- Methods:
 - constructor: create a new book
 - borrow_book(): marks as borrowed
 - return_book(): marks as not borrowed
 - get_status(): returns "Available" or "Borrowed"

Library Class

- Attributes:
 - a collection of Book objects
- Methods:
 - add_book(book): Adds a new book to the catalog.
 - borrow_book(id): borrow a book, if it is already borrowed, print "Not available".
 - return_book(id): return a book.
 - list_available_books(): print information of all available books in id increasing order
 - support indexing: Library[i] returns the i-th book added to the library



Inheritance

Inheritance is a mechanism that allows a *newly class(subclass)* to acquire the attributes and methods of an existing class (*parent class* or *base class*)

- Provides a hierarchical way to organize and structure code

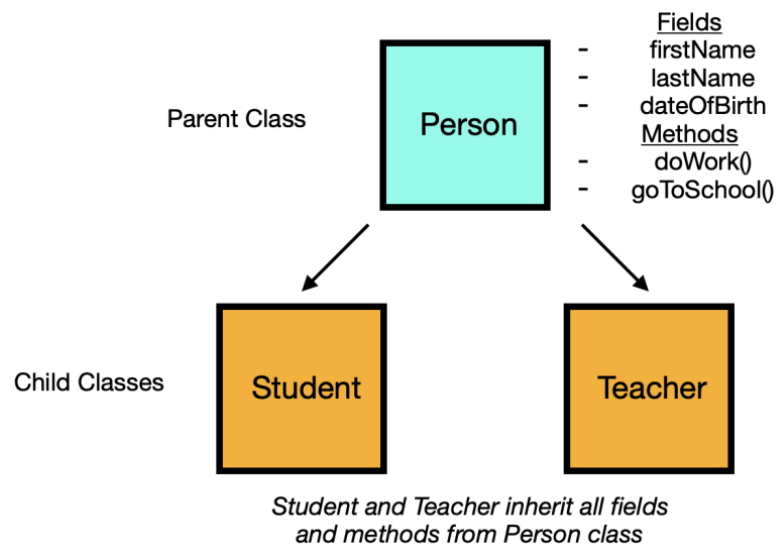
Features

- **Reusability**
 - Subclasses can directly reuse the code of the parent class
- **Extensibility**
 - Subclasses can define new attributes and methods
 - Parent class methods can be **overridden** to provide more specific functionalities

Inheritance

Example:

- A person has attributes like names, location, and methods like eat(), and drink()
- Both 'teachers' and 'students' are subclasses of 'person' and therefore inherit these attributes and methods
- Subclasses can extend functionality



Parent Class (super class)

- The class being inherited from
- ## Child class (sub class)
- The class inherits from the parent class

Inheritance

Syntax: `class SubClassName(ParentClassName)`

Subclass inherits all attributes and methods from its parentclass

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am a human from {}".format(self.location))
```

```
Sub class   Parent class
class Student(Human):
    def __init__(self, id):
        self.id = id
stu = Student(id='HKUST(GZ)-456')
stu.introduce()
print(stu.id)
```

✓ 0.0s

```
I am a human from Guangzhou.
HKUST(GZ)-456
```

Inheritance

The initializer of the superclass is **not** called automatically in a subclass

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
        print('Human constructed')
```

```
class Student(Human):
    def __init__(self, id):
        self.id = id
stu = Student(id='HKUST(GZ)-456')
stu.introduce()
print(stu.id)
```

⊗ 0.0s

AttributeError Traceback (most recent call last)

Cell In[95], line 5

```
3         self.id = id
4     stu = Student(id='HKUST(GZ)-456')
----> 5 stu.introduce()
6     print(stu.id)
```

Cell In[94], line 6

```
4 def introduce(self):
5     print("I am {} from {}. I am {} years old. "\
----> 6         .format(self.name, self.location,
7                 self.age))
```

AttributeError: 'Student' object has no attribute 'name'

Inheritance

Call the superclass's initializer explicitly

SuperClass.__init__(self, arguments)

- The current object must be explicitly passed using self

```
class Student(Human):  
    def __init__(self, name, age, id):  
        Human.__init__(self, name, age)  
        self.id = id  
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')  
alice.introduce()  
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed  
I am Alice from Guangzhou. I am 22 years old.  
Alice 22 HKUST(GZ)-456
```

Inheritance

Call the superclass's initializer explicitly

super(ClassName, object).__init__(arguments)

- Super object: returns a special object that allows access to the superclass' functions (the superclass of *ClassName*)
- *super()*: equals `super(ClassName, self)` in Python 3

```
class Student(Human):
    credits = 0
    def __init__(self, name, age, id):
        super(Student, self).__init__(name, age)
        self.id = id
alice = Student('Alice', 22, id='HKUST(GZ)-456')
alice.introduce()
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed
I am Alice from Guangzhou. I am 22 years old.
Alice 22 HKUST(GZ)-456
```

```
class Student(Human):
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
alice.introduce()
print(alice.name, alice.age, alice.id)
```

✓ 0.0s

```
Human constructed
I am Alice from Guangzhou. I am 22 years old.
Alice 22 HKUST(GZ)-456
```

Inheritance

Subclasses can define additional attributes and methods beyond those inherited from the superclass

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
class Student(Human):
    credits = 0

    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def gain_credit(self, credits):
        self.credits += credits

    def print_credit(self):
        print("I have {} credits".format(self.credits))

alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
alice.introduce()
alice.gain_credit(10)
alice.print_credit()
```

Inheritance

A subclass can override methods defined in its superclass

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

✓ 0.0s

Priority of calling

- Call the current class's method by default
- Call the super class's method if the current class does not have that method

```
class Student(Human):
    credits = 0

    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am a student from HKUST-GZ. My name is {}. I am {} years old. "\
              .format(self.name, self.age))

    def gain_credit(self, credits):
        self.credits += credits

    def print_credit(self):
        print("I have {} credits".format(self.credits))
```

```
alice = Student(name='Alice', age=22, id='HKUST(GZ)-456')
```

```
alice.introduce()
```

```
super(Student, alice).introduce()
```

✓ 0.0s

```
I am a student from HKUST-GZ. My name is Alice. I am 22 years old.
```

```
I am Alice from Guangzhou. I am 22 years old.
```

Polymorphism

- Polymorphism refers to different objects implementing the same method name in their own way.
- A powerful feature of OOP, often built on top of inheritance

```
class Human:
    location = 'Guangzhou'

    def introduce(self):
        print("I am {} from {}. I am {} years old. "\
              .format(self.name, self.location,
                      self.age))

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

✓ 0.0s

```
class Student(Human):
    credits = 0
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am a student from HKUST-GZ. My name is {}. I am {} years old. "\
              .format(self.name, self.age))

class Professor(Human):
    credits = 0
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.id = id

    def introduce(self):
        print("I am {}, a professor in HKUST-GZ.".format(self.name))

alice = Student('Alice', 20, 'HKUST(GZ)-456')
alice.introduce()
bob = Professor('Bob', 40, 'GZ-0000123')
bob.introduce()
```

✓ 0.0s

I am a student from HKUST-GZ. My name is Alice. I am 20 years old.
I am Bob, a professor in HKUST-GZ.

Task: Create a base class named Animal and derive two classes from it: Bird and Mammal. Each class will have attributes and methods that demonstrate the concepts of inheritance

Base Class - Animal

- **Attributes:** name (str), age (int)
- **Methods:**
 - `__init__(self, name, age)`
 - `make_sound(self)`: Prints a generic sound.

Derived Classes

- **Bird**

- **Unique Attribute:** wing_span - the wing span in meters.
- **Method Override:**
 - `__init__(self, name, age, wing_span)`
 - `make_sound(self)`: print "Chirp".

- **Mammal**

- **Unique Attribute:** fur_color (str) - the color of the fur.
- **Method Override:**
 - `__init__(self, name, age, fur_color)`
 - `make_sound(self)`: print "Roar".



Modules and Packages

What is a Module

A file containing Python definitions (e.g, functions, classes) and statements, ending in .py

The file name becomes the module name

Motivation:

- **Organization:** Group related functions and classes together.
- **Reusability:** Use the same code across multiple projects without copy-pasting
- **Namespacing:** Avoid name collisions between different parts of your code.
`my_module.my_func()` won't conflict with `other_module.my_func()`
- Built-in Modules: math, string, date, random etc..
- User-Defined Modules: Just save the source code file with extension **.py**

Use Module: import statement

Syntax	Notes
<code>import module</code>	Best practice. Access via <code>module.name</code> .
<code>import module as m</code>	Good for long names (e.g., <code>numpy as np</code>).
<code>from module import a, b</code>	Brings specific names into your scope.
<code>from module import *</code>	Avoid! Can cause name collisions.

```
1 # import the whole module
2 import math
3 print(math.pi)
4 # import selected names
5 from math import sqrt, sin
6 print(sqrt(9)) # Don't need math.sqrt()
7 # alias a module
8 import numpy as np
9 arr = np.array([1, 2, 3])
```

User-defined Module Example

Let's create our own module to hold AI utility functions.

File: ai_utils.py

```
1 # ai_utils.py
2 import numpy as np
3 def softmax(x):
4     """Compute softmax
5     values for each
6     sets of scores in x
7     ."""
8     e_x = np.exp(x - np.
9         max(x))
10    return e_x / e_x.sum()
11 VERSION = "1.0"
```

File: main.py (in same folder)

```
1 # main.py
2 import ai_utils
3 import numpy as np
4 logits = np.array([2.0,
5     1.0, 0.1])
6 probabilities = ai_utils.
7     softmax(logits)
8 print(f"Probs: {
9     probabilities}")
10 print(f"Using utils v{
11     ai_utils.VERSION}")
```

Result: We've successfully separated our logic. main.py handles the high-level task, while ai_utils.py provides a reusable component

■ `__name__ == '__main__'`

- Python sets a special variable `__name__` for every module
- If a file is executed directly (e.g., `python my_module.py`), its `__name__` is set to `"__main__"`.
- If the file is imported, its `__name__` is set to the module's name (e.g., `"my_module"`)
- This allows you to write code that only runs when the file is executed directly. This is the standard way to include tests or examples for your module

```
1 # in ai_utils.py
2 def softmax(x):
3     # ... (implementation from before) ...
4 # This block only runs when you execute "python ai_utils.py"
5 if __name__ == "__main__":
6     print("Running a test of the softmax function...")
7     test_data = np.array([3.0, 1.0, 0.2])
8     test_result = softmax(test_data)
9     print(f"Test input: {test_data}")
10    print(f"Test output: {test_result}")
11    assert np.isclose(np.sum(test_result), 1.0)
12    print("Test passed!")
```

From Modules to Packages

- When a project grows, you might need more organization than just a few .py files.
- A Python package is simply a directory of modules.
- A common structure for an AI project might look like this:
 - The `__init__.py` files can be empty.
 - They tell Python that the directory is a package.

```
my_ai_project/  
|-- main.py  
|-- data/  
|   |-- __init__.py  
|   |-- loaders.py  
|-- models/  
|   |-- __init__.py  
|   |-- cnn.py  
|-- utils/  
    |-- __init__.py  
    |-- metrics.py
```

■ ■ ■ Importing from Packages

- With a package structure, you use dot notation to import.

Assume you are running main.py:

```
1  # in main.py
2  # Absolute imports from the project root
3  from data.loaders import load_image_data
4  from models.cnn import SimpleCNN
5  from utils.metrics import calculate_f1_score
6  # --- Or using aliases ---
7  import utils.metrics as metrics
8  # ... code that uses these imported functions ...
9  model = SimpleCNN()
10 images = load_image_data('path/to/images')
11 # ...
12 f1 = metrics.calculate_f1_score(preds, labels)
```

This structure is scalable and used by all major AI libraries like scikit-learn, tensorflow, and pytorch.

Task: Quickly structure a Python project by creating a module and organizing it into a package.

1. Create & Use Module:

- Make file `data_utils.py` with a `normalize()` function.
- Create `main.py` that imports and calls this function.

2. Add Test Guard:

- In `data_utils.py`, add an `if __name__ == "__main__":` block to self-test your `normalize()` function.

3. Build Package & Fix Import:

- Restructure your files into the package layout shown below (create folders and empty `__init__.py`).
- Then, fix the import statement in `main.py` to work with the new path

```
my_project/  
|-- main.py  
|-- utils/  
    |-- __init__.py  
    |-- data_utils.py
```



Advanced Iteration: Recursion & Generators

What is Recursion?

- Recursion is a programming technique where a function calls itself in order to solve a problem that can be broken down into smaller, similar sub-problems.
- A recursive function must have two parts:
 - **Base Case:** A simple condition that stops the recursion. Without a base case, you get infinite recursion (and a `RecursionError`).
 - **Recursive Step:** The part of the function that calls itself, but with inputs that move it closer to the base case.
- Analogy: Russian Nesting Dolls. Each doll contains a smaller version of itself, until you reach the smallest doll that cannot be opened (the base case).

Classic Example: Factorial

The recursive definition of factorial $n!$:

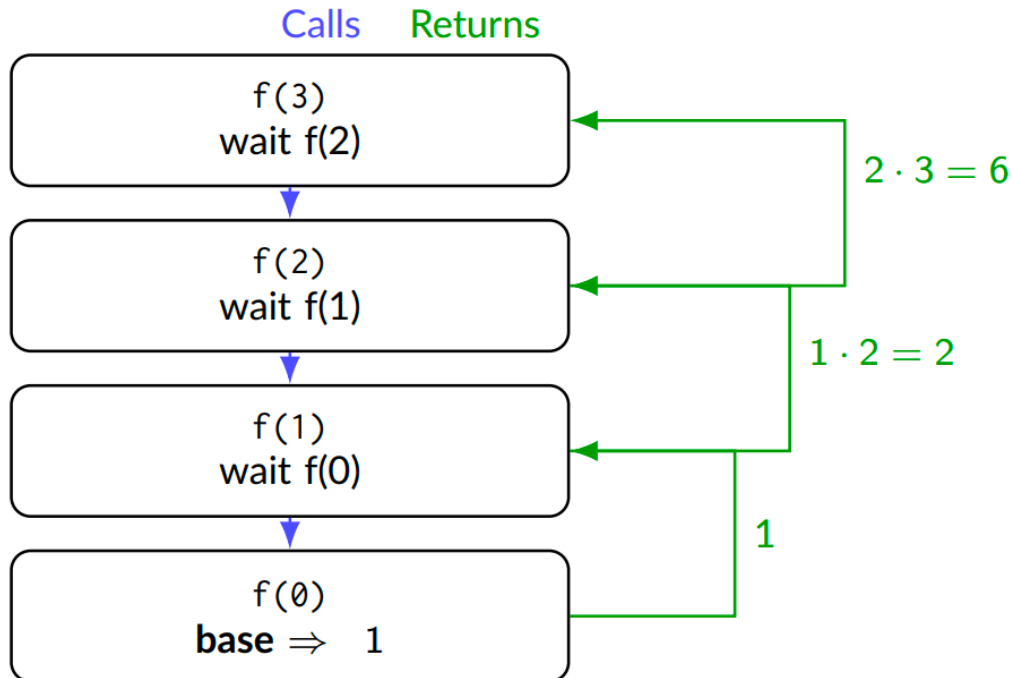
$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ (Base Case)} \\ n \times (n - 1)! & \text{if } n > 0 \text{ (Recursive Step)} \end{cases}$$

```
1 def factorial(n):
2     # Base Case:
3     if n == 0:
4         return 1
5     # Recursive Step:
6     else:
7         return n * factorial(n - 1)
8
9 print(factorial(4)) # 24
```

Visualizing Recursion: The Call Stack

How factorial(3) runs:

expansion then contraction



Compact Trace

- 1) $f(0) \Rightarrow 1$
- 2) $f(1) = 1 \cdot f(0) \Rightarrow 1$
- 3) $f(2) = 2 \cdot f(1) \Rightarrow 2$
- 4) $f(3) = 3 \cdot f(2) \Rightarrow 6$

Mental Model

- Expansion: push frames until base case.
- Contraction: pop frames, combine results.
- Each frame has its own n ; they don't interfere.

Checkpoints

- Base case reachable? (stops recursion)
- Progress toward base? ($n-1$)
- Work on return path? (e.g., $n * f(n-1)$)

Pros

- **Clear for Recursive Problems:** Naturally recursive tasks (trees, graphs, parsing) can be expressed in a simple, readable way.
- **Simplifies Thinking:** Avoids complex loops by breaking a problem into self-similar pieces.

Cons

- **More Memory & Time:** Each call adds a new stack frame, which can be slower than iteration.
- **Depth Limit Risk:** Very deep recursion may hit Python's call stack limit (RecursionError).

Guideline: Use recursion when it makes code clearer. For very deep or simple loops, prefer iteration.

Generator and the yield keyword

A generator is a special function that returns a lazy iterator. It produces items one at a time and only when requested.

- Instead of return, a generator function uses **yield**.
- When yield is hit, the function's state is "frozen", a value is sent to the caller, and it waits.
- When the next value is requested (e.g., in a for loop), the function resumes from where it left off

```
1 def count_up_to(max_val):
2     print("Generator starting...")
3     count = 1
4     while count <= max_val:
5         yield count # Pauses here and sends 'count' back
6         count += 1
7     print("Generator finished.")
8 # A for loop automatically handles the iteration
9 for number in count_up_to(3):
10    print(f"Received {number}")
```

Motivation: Why Generators?

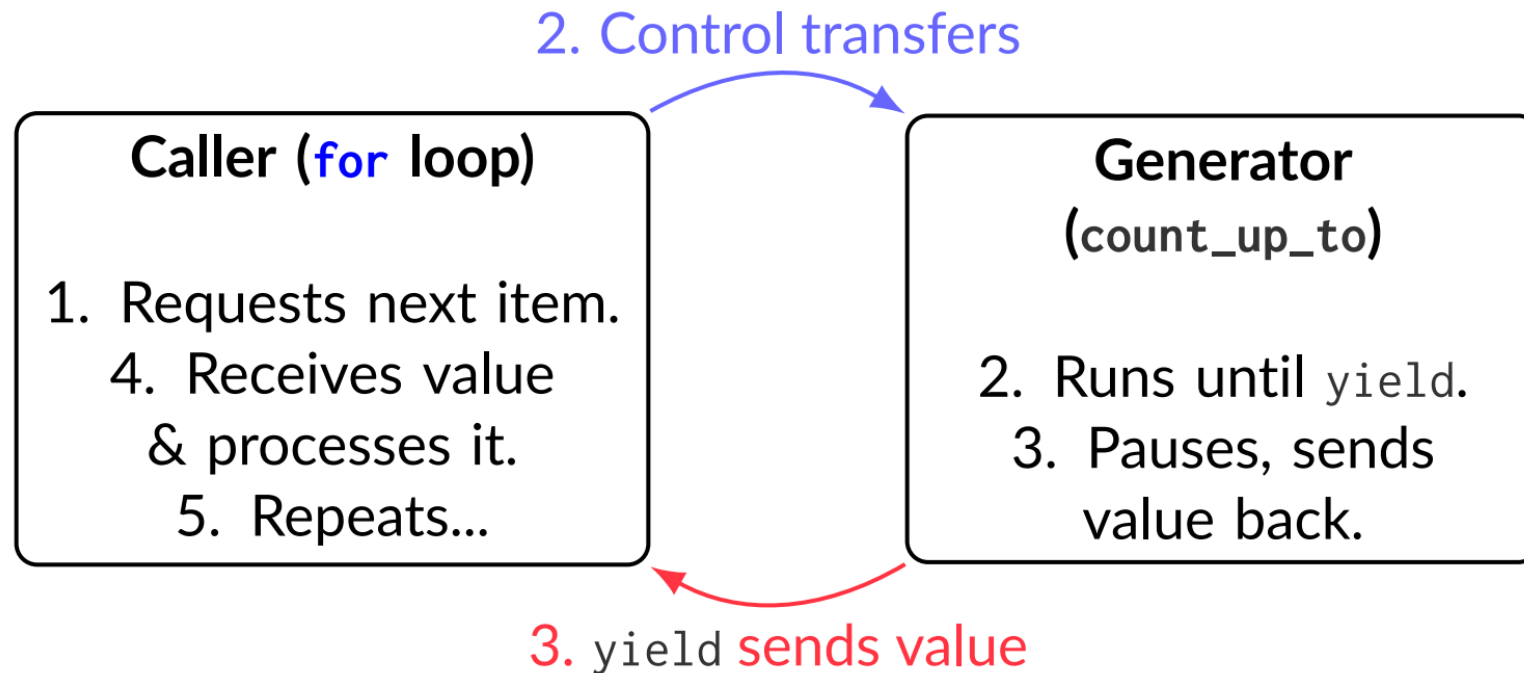
- Large datasets (e.g., 50 GB logs, millions of images) cannot be loaded at once. Reading everything eagerly will exhaust memory.
- Generators enable lazy, memory-efficient processing: produce one item at a time and keep RAM usage small.

```
1 # Avoid eager loading of huge files
2 def read_huge_file(path):
3     with open(path, 'r') as f:
4         return f.readlines() # Loads all lines into RAM
5                               -- risky
6 all_data = read_huge_file('large_dataset.csv') # Likely
           to crash
```

Visualizing Generators: The yield Dance

- This cycle repeats until the generator finishes, at which point it raises a `StopIteration` exception internally, which cleanly ends the `for` loop.

How a `for` loop interacts with a generator:



■ ■ ■ From Recursion to Generators

When recursion fits

- Tree/graph traversal; divide-and-conquer where a depth-first structure is natural.

Where it struggles

- Long, linear scans over huge data: stack depth and repeated function calls increase memory use and execution time.

Enter generators

- Lazy, streaming iteration: produce items on demand with $O(1)$ extra memory.
- Compose pipelines (map/filter) without building full lists.

Rule of thumb

- recursive shape $\Rightarrow$ recursion
- massive streams $\Rightarrow$ generators

AI Application: Data Batch Generator

- In deep learning, we train models on batches of data, not the entire dataset at once.
- Generators are the perfect tool for this memory-efficient task.
- This approach generates one batch at a time, without preparing all batches in advance.

```
1 import numpy as np
2 def batch_generator(features, labels, batch_size):
3     """Yields batches of data for training."""
4     num_samples = len(features)
5     # Loop forever to allow for many epochs of training
6     while True:
7         indices = np.random.permutation(num_samples)
8         for i in range(0, num_samples, batch_size):
9             batch_indices = indices[i : i + batch_size]
10            yield features[batch_indices], labels[batch_indices]
11 # Usage with a large dataset:
12 # features, labels = load_my_huge_dataset()
13 # train_gen = batch_generator(features, labels, batch_size=32)
14 # for i in range(num_training_steps):
15 #     batch_x, batch_y = next(train_gen)
16 #     model.train_on_batch(batch_x, batch_y)
```

Task: Use recursion and generators for common data tasks and compare their strengths.

1. **Recursive Search:** Write a recursive function `find_files(fs_dict, ext)` to find all file paths in a nested dictionary that end with a given extension `ext`
2. **Memory-Efficient Log Scan:** Write a generator function `get_errors(log_path)` to yield only the lines starting with "ERROR" from a potentially huge log file, keeping memory usage low.
3. **Compare Approaches:**
 - Explain: Why is recursion a natural fit for Task 1?
 - Explain: Why would recursion be a poor choice for Task 2, especially with millions of log entries?

Classes and Objects

- Objected-Oriented Programming, Encapsulation
- Constructor/Destructor, Class/Object-level attributes
- Dunder Methods (`__lt__`, `__str__`, `__len__`, `__getitem__`)

Inheritance

- Subclass/Superclass, Polymorphism

Modules and Packages

- built-in modules, user-defined modules, import
- `__name__ == '__main__'`, packages

Advanced Iteration: Recursion & Generators

- yield statement

Python Programming with Artificial Intelligence

Thank you for listening!

